SoC101 | 2026-1

Session6 : Sequential Logic Blocks(1)

Agenda

- Basic Sequential Blocks
  - Latch & Flip-Flop
    - D-Latch
    - D-Flip Lop
  - Counter and Shift Register
- LABS

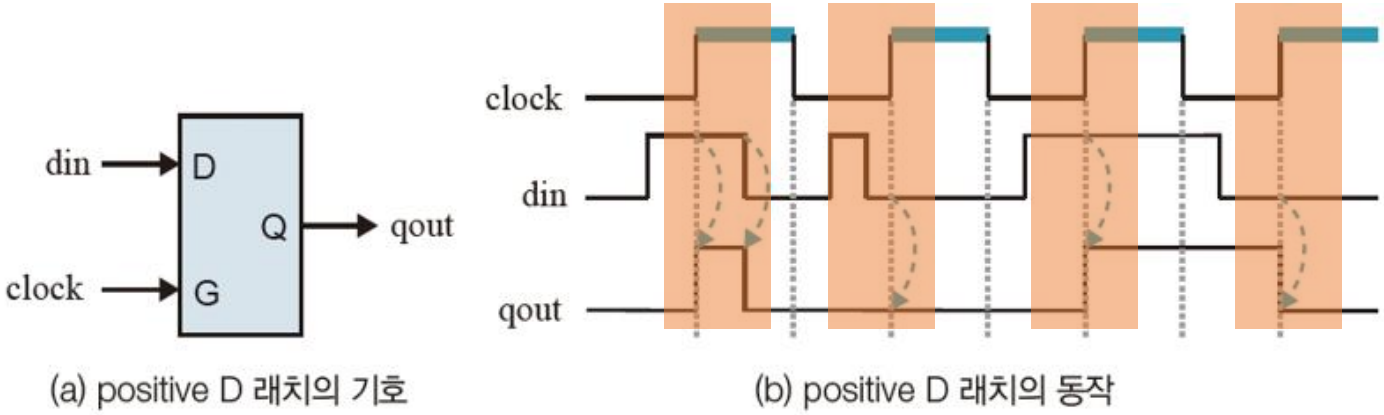
1. Latch and Flip Flop

Sequential Logic

- 순차회로
  - 현재의 입력, 과거의 입력, 회로에 기억된 상태값에 의해 출력이 결정
  - 과거의 입력, 현재의 상태값을 저장하는 저장소자(래치, 플립플롭)와 조합논리회로로 구성
  - 데이터 레지스터, 시프트 레지스터, 계수기(counter), 직렬/병렬 변환기, 유한상태머신(Finite State Machine; FSM), 주파수 분주기, 펄스 발생기 등
- 래치와 플립플롭
  - 래치 (latch) : 클록신호의 **레벨**(즉, 0 또는 1)에 따라 동작하는 저장소자
  - 플립플롭 (Flop-flop) : 클록신호의 **상승 또는 하강에지**에 동기되어 동작하는 저장소자
  - always 구문 내부에 if 조건문을 이용하여 모델링
- 순차회로의 모델링
  - always 블록을 이용한 행위수준 모델링, 게이트 프리미티브 및 하위모듈 인스턴스, 연속 할당문 등 다양한 Verilog HDL 구문들이 사용됨
  - 할당문의 형태 (nonblocking 또는 blocking)에 따라 회로의 동작과 구조가 달라짐

D-Latch

- **클록 신호의 레벨 (0 또는 1)에 따라 통과모드 또는 유지모드로 동작**
  - 통과 (transparent) 모드 : 입력 D가 출력 Q로 통과됨(Latch가 열려있음)
  - 유지 (hold) 모드 : 출력 Q의 값이 유지됨(Latch가 닫혀있음)
  - positive (level-sensitive) latch, negative (level-sensitive) latch
  - Latch의 Timing 문제 : 통과모드에서 입력이 빠르게 변할 경우 출력이 불안정해 질 수 있음



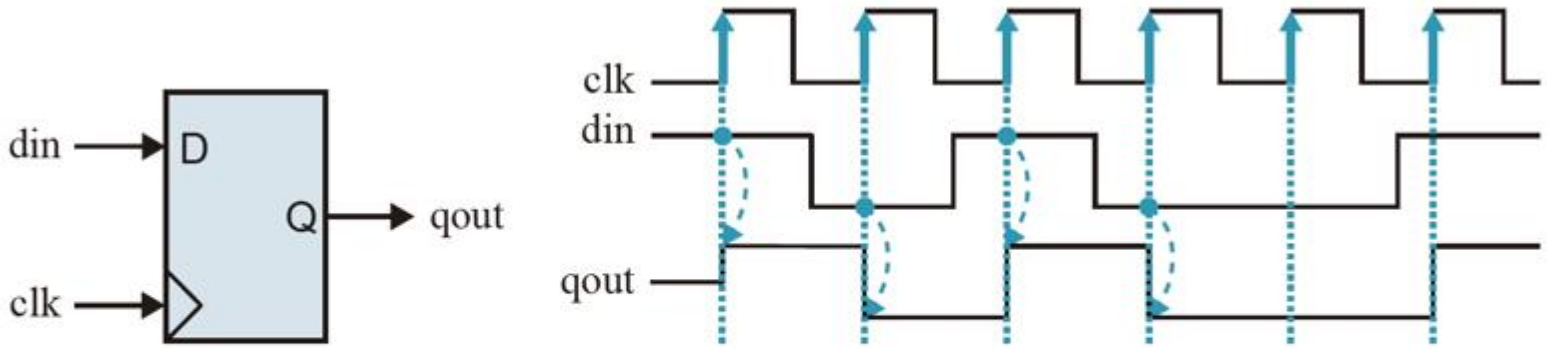
(a) positive D 래치의 기호

(b) positive D 래치의 동작

[그림 11-1] positive D 래치

D Flip-Flop

- **클록 신호의 엣지변화 (transition edge)에서 동작**
  - 한 클록 주기 동안 정보를 저장함
  - positive edge-triggered(0 → 1), negative edge-triggered(1 → 0)
  - always 블록의 감지신호목록(sensitivity list)에 클록 신호만 포함
  - 클록신호의 특정 edge에서만 입력을 받아들이기 때문에 glitch에 강하고 안정적인 동작을 보장 → 시스템 타이밍 예측 용이, 회로 안정성



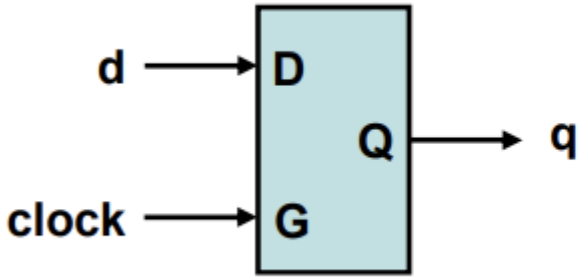
(a) D 플립플롭 (b) 상승에지 트리거 D 플립플롭의 동작

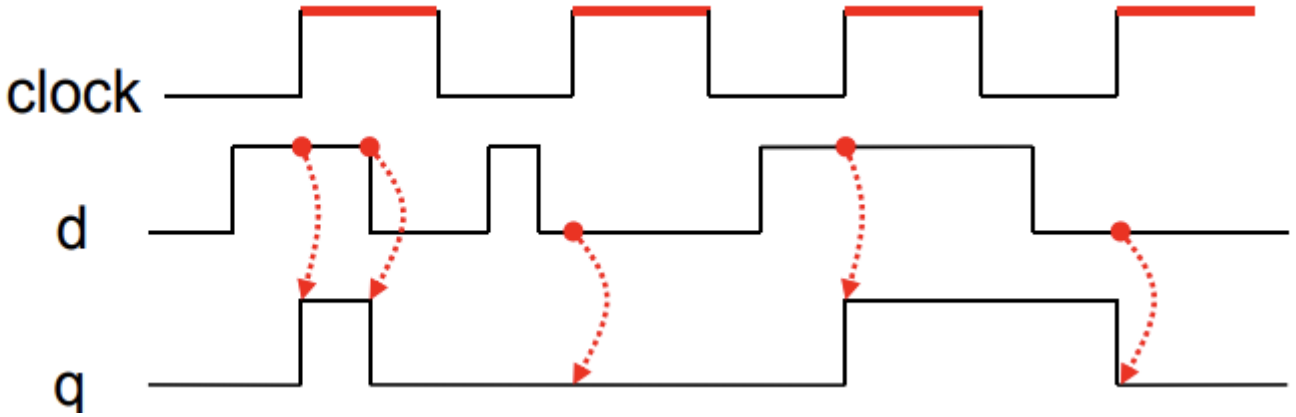
[그림 11-6] 상승에지 트리거 D 플립플롭

Comparison btw Latch and FF

특성	래치	플립플롭
클록 감응성	레벨에 반응	에지에 반응
데이터 변경	클록 활성 레벨 동안 지속적	클록 에지에서만
안정성	글리치에 취약	글리치에 강함
동기화	비동기 동작 가능	주로 동기식 시스템에서 사용
회로 복잡성	상대적으로 단순	더 복잡 (일반적으로 두 개의 래치로 구성)
전력 소비	상대적으로 낮음	상대적으로 높음
주요 응용	간단한 저장, 비동기 시스템	동기식 디지털 시스템, 레지스터, 카운터
Verilog 모델링	always @(*) 또는 always @(CLK)	always @(posedge CLK) 또는 always @(negedge CLK)

LAB1 : D-Latch Specification





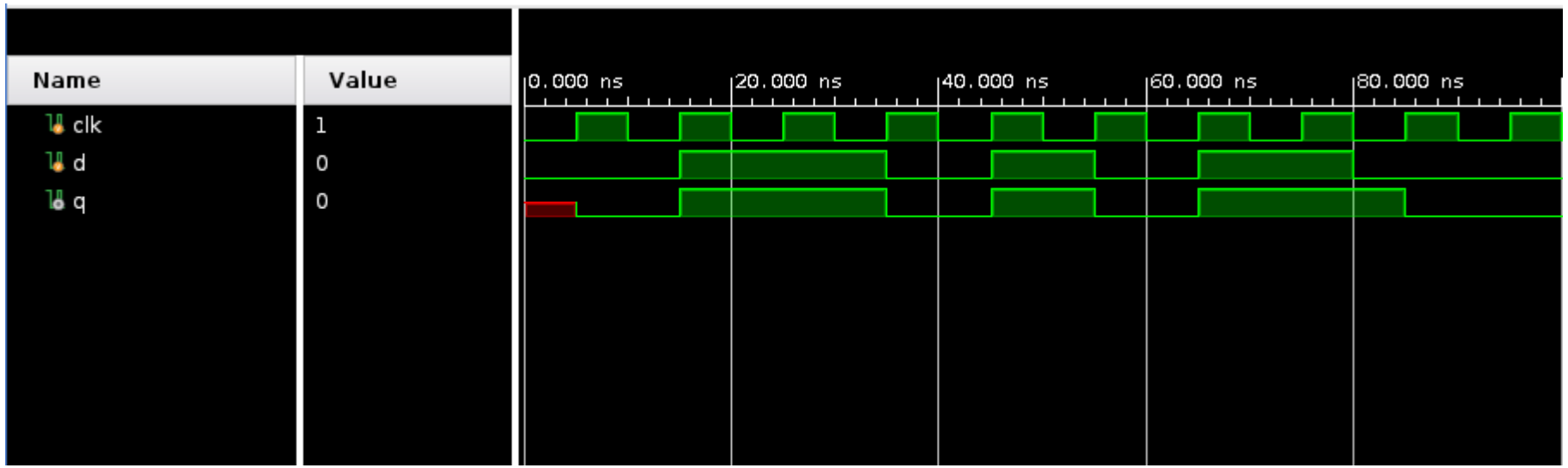
LAB1 : DUT

```
`timescale 1ns/1ps

module d_latch (
    // port list
    clk,
    d,
    q
);

input clk;
input d;
output reg q;

always @(*) begin
    if( clk ) q <= d;
end

endmodule
```

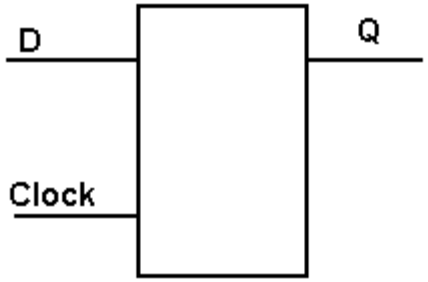
LAB1 : Testbench

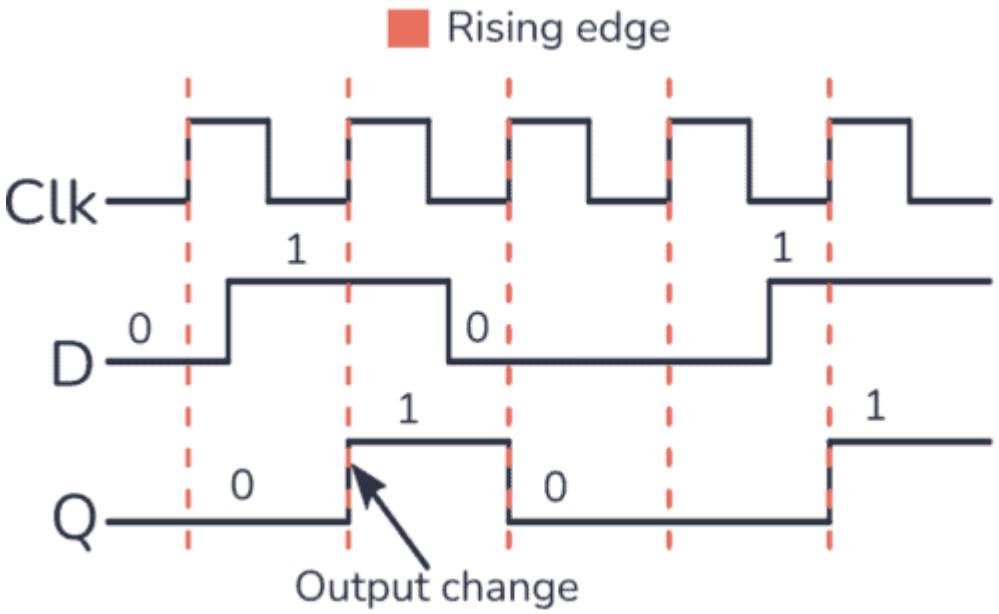
```
`timescale 1ns/1ps

module tb_d_latch (
    // TB
);

// signal def
reg clk, d;
wire q;

// DUt instantiation
d_latch DUT(
    .clk(clk),
    .d(d),
    .q(q)
);

// clock gen
always #5 clk = ~clk; // 10ns

// put stimulus

initial begin
    clk = 1'b0;
    d = 1'b0;

    #15 d=1'b1; #20 d=1'b0;
    #10 d=1'b1; #10 d=1'b0;
    #10 d=1'b1; #15 d=1'b0;

    #50; $finish;

end
endmodule
```

LAB1 : Waveform



LAB2 : D Flip Flop Specification





LAB2 : DUT

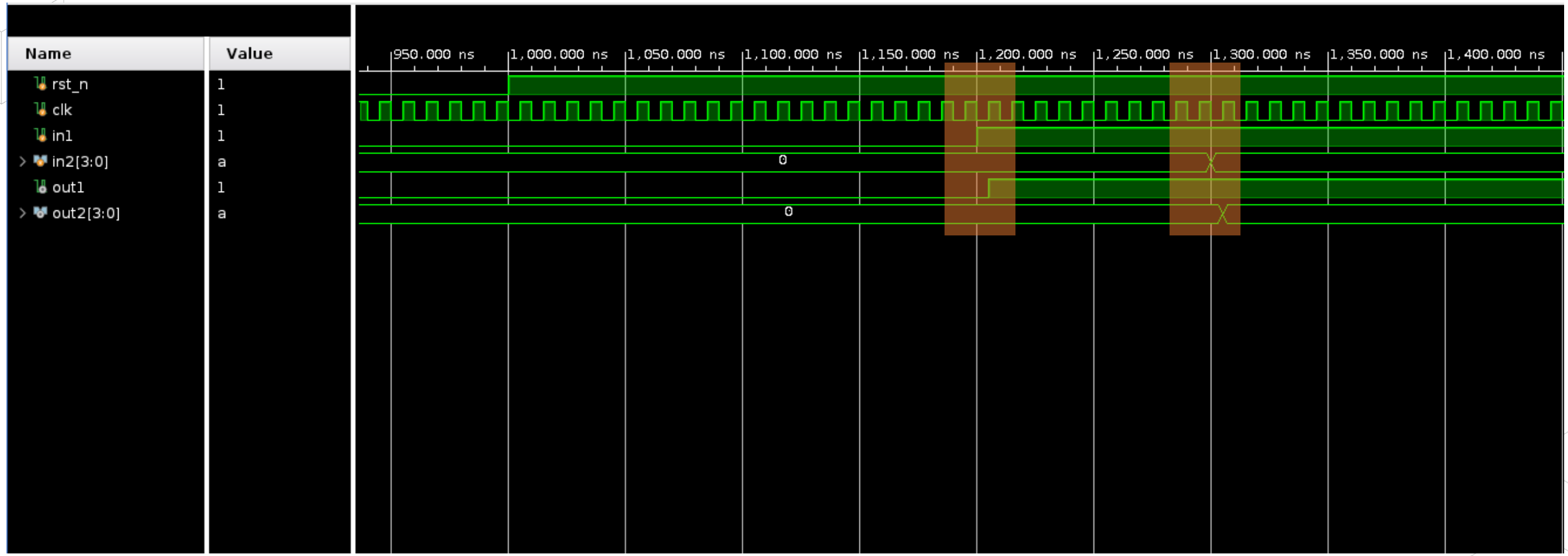
```
`timescale 1ns / 1ps

module register (
    rst_n,
    clk,
    in1, // 1-bit input
    in2, // 4-bit input
    out1, // 1-bit input
    out2 // 4-bit output
); // port list

// port declaration and IO direction
input wire rst_n;
input wire clk;
input wire in1;
input wire [3:0] in2; // 4bit input
output reg out1;
output reg [3:0] out2; // 4bit output


// behavioral modeling: 1-bit register
// reg out1;
always @(posedge clk or negedge rst_n)
begin
    if ( ~rst_n ) out1 <= 1'b0; // active low | rst enable
    else out1 <= in1;
end



// behavioral meodeling: 4-bit register
//reg [3:0] out2;
always @(posedge clk, negedge rst_n)
begin
    if ( rst_n == 0 ) out2 <= 4'b0;
    else out2 <= in2;
end

endmodule
```

LAB2 : Testbench

```
1   `timescale 1ns / 1ps
2
3   module tb_register (); // top module
4
5       // signal declaration
6       reg         rst_n, clk      ;
7       reg         in1             ;
8       reg [3:0]   in2             ;
9       wire        out1            ;
10      wire [3:0]  out2            ;
11
12      /***** inject stimulus *****/
13
14      initial begin
15      // initial value config(t = 0)
16      rst_n       = 1'b0      ; // active low rst
17      clk         = 1'b0      ;
18      in1         = 1'b0      ;
19      in2         = 4'b0000   ;
20
21
22      // t = 1000ns, rst release
23      #1000 rst_n = 1'b1;
24
25      // t = 1200ns
26      #200 in1 = 1'b1;
27
28      // t = 1300ns
29      #100 in2 = 4'b1010;
30
31      // t = 1800ns, rst enable: erase all value if registers
32      #500 rst_n = 1'b0;
33
34      // t = 2000ns
35      #200 in1 = 1'b0;
36           in2 = 4'd0;
37
38      // t = 2300ns
39      #300 in1 = 1'b1;
40           in2 = 4'b1111;
41      // t=2400
42      #100 rst_n = 1'b1;
43
44      // t = 2500ns, terminate sim
45      #200 $finish;
46      end
```

```
47
48      /***** clkgen *****/
49      always #5 clk = ~clk;
50
51
52      /***** DUT instantiation *****/
53      register DUT
54      (
55      .rst_n  ( rst_n ),
56      .clk    ( clk   ),
57      .in1    ( in1   ),
58      .in2    ( in2   ),
59      .out1   ( out1  ),
60      .out2   ( out2  )
61
62      );
63  endmodule
```

LAB2 : Waveform



2. Counter

Counter

- 계수기(counter)
  - 클록 펄스가 인가될 때마다 값이 증가 또는 감소되는 회로
  - 주파수 분주기, 타이밍 및 제어 신호 생성 등 디지털 회로 설계에 폭넓게 사용

동기식 계수기

- 모든 플립플롭이 **하나의 공통 클록신호**에 의해 구동되며, 모든 플립플롭의 상태변경이 동시에 일어남
- 장점 : 설계와 검증이 용이하며, 계수 속도가 빠름
- 단점 : 비동기식 카운터에 비하여 회로가 복잡함

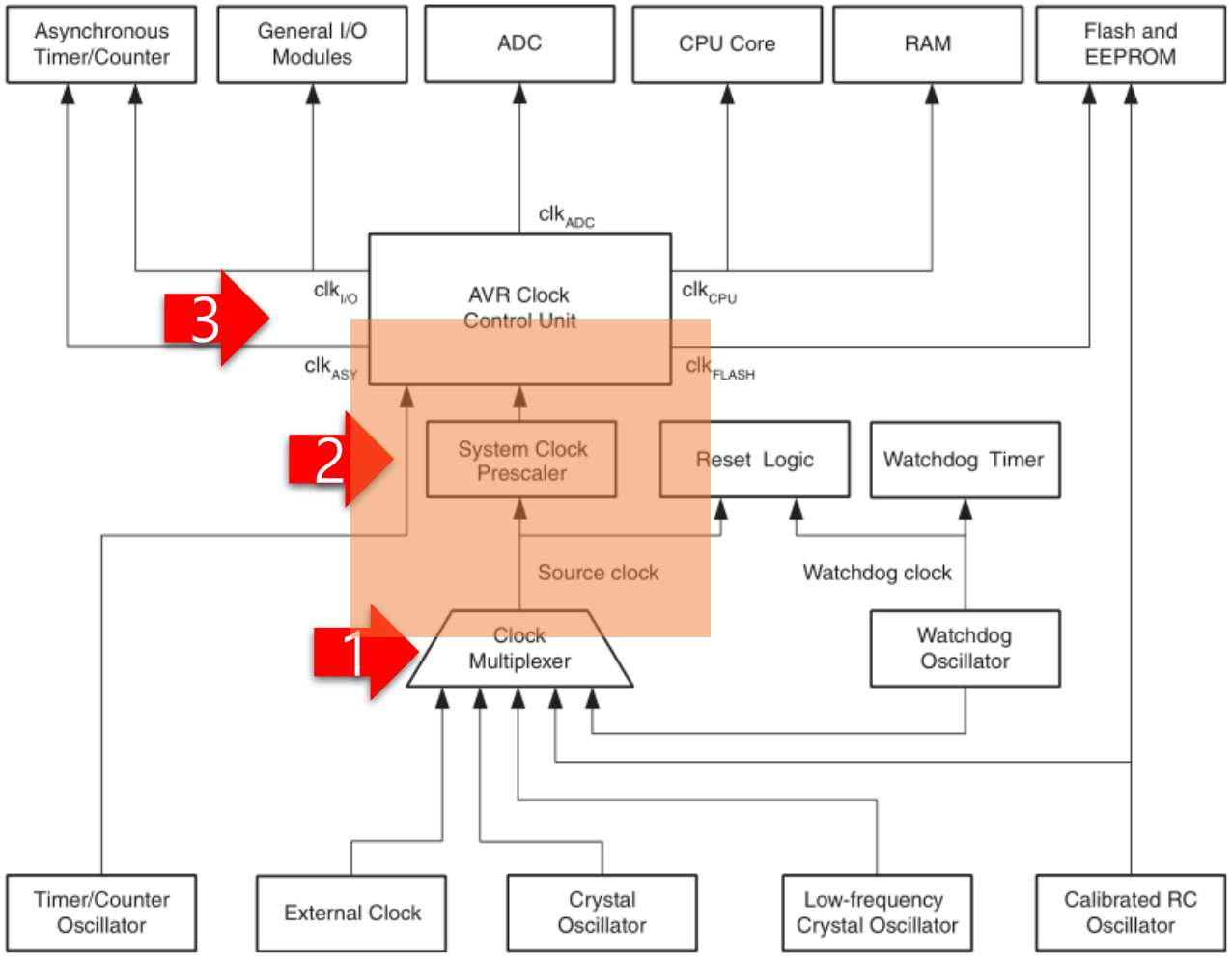
비동기식 계수기

- 첫단의 플립플롭에 클록신호가 인가되면, 플립플롭의 출력이 다음 단의 플립플롭을 트리거시키는 방식으로 동작 → 리플 계수기(ripple counter)라고도 함
- 장점 : 동기식 계수기에 비해 회로가 단순해짐
- 단점 : 각 플립플롭의 전파 지연이 누적되어 최종단의 출력까지 전파되어 속도가 느림

Counter

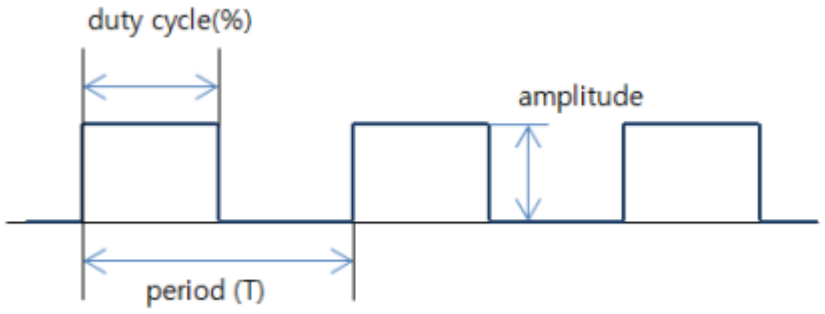
- SoC 주변장치에서의 활용
  1. 주파수 분주: 높은 주파수의 클록 신호를 낮은 주파수로 나누는 데 사용
  2. 타이밍 생성: 특정 시간 간격으로 이벤트를 트리거하는 데 사용
  3. 제어 신호 생성: 다른 주변장치나 모듈을 제어하기 위한 신호를 생성
  4. 이벤트 카운팅: 특정 이벤트의 발생 횟수를 세는 데 사용

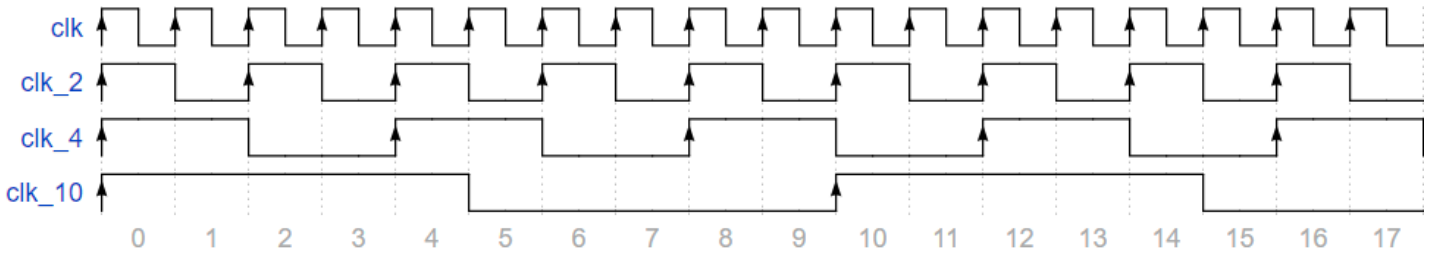
주파수 분주기 사용 예



LAB : Counter Specification

- frequency 1/10 pre-scaler , duty 50%





주파수 분주기

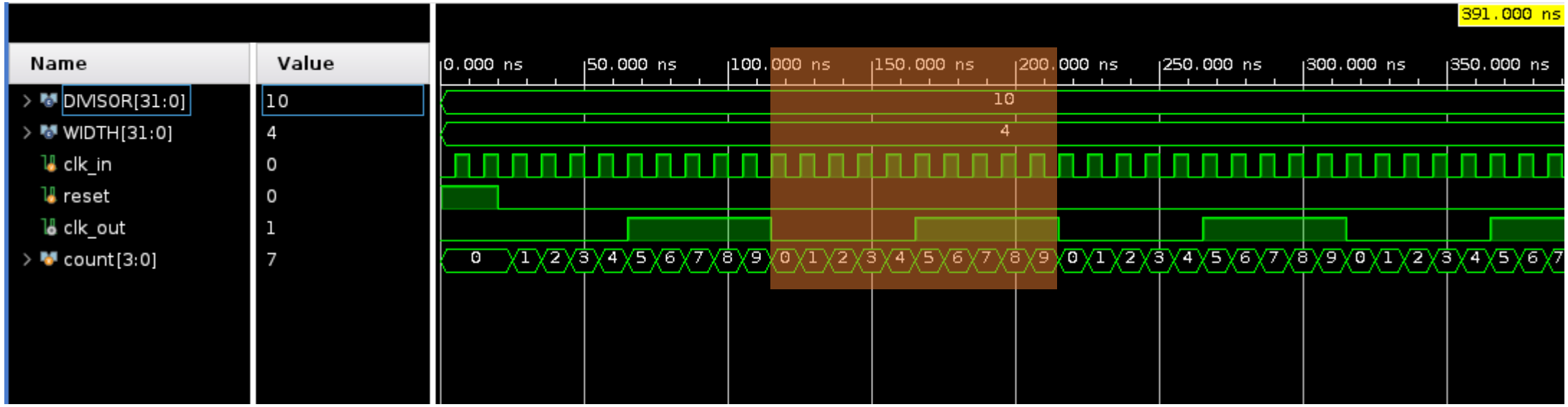
LAB : DUT

```
`timescale 1ns / 1ps
module fdivider #(
    parameter DIVISOR = 10
)(
    input wire clk_in,
    input wire reset,
    output reg clk_out
);

    localparam WIDTH = $clog2(DIVISOR); // calc the minimum number of bits required to represent a given number
                                        // e.g) To represent 9 values, we nned
                                        // at least 4bits since 2^4 = 16 >= 9,
                                        // so clog2(9) = 4
    reg [WIDTH-1:0] count;

    always @(posedge clk_in or posedge reset) begin
        if (reset) begin
            count <= 0;
            clk_out <= 1'b0;
        end else begin
            if (count == DIVISOR - 1) begin
                count <= 0;
            end else begin
                count <= count + 1'b1;
            end

            // DIVISOR의 절반에 도달할 때마다 출력 토글
            if (count == (DIVISOR / 2) - 1) begin
                clk_out <= 1'b1;
            end else if (count == DIVISOR - 1) begin
                clk_out <= 1'b0;
            end
        end
    end

endmodule
```

LAB : Testbench

```
`timescale 1ns / 1ps
module tb_fdivider;
    // signal list
    reg clk_in;
    reg reset;
    wire clk_out;

    // DUT instantiation with the designated param
    fdivider #(
        .DIVISOR(10)
    ) divider (
        .clk_in(clk_in),
        .reset(reset),
        .clk_out(clk_out)
    );

    // clock gen (10 ns, 100MHz)
    always #5 clk_in = ~clk_in;

    // apply stimulus
    initial begin
        clk_in = 0;
        reset = 1;
        #20 reset = 0;  // 20ns 후 리셋 해제
        // 200 사이클 동안 시뮬레이션
        repeat(200) @(posedge clk_in);
        $finish;
    end
    // Monitor the results
    always @(posedge clk_in) begin
        $display("Time=%0t, clk_in=%b, clk_out=%b", $time, clk_in, clk_out);
    end
endmodule
```

LAB : Waveform



3. Shift Register

Shift Register

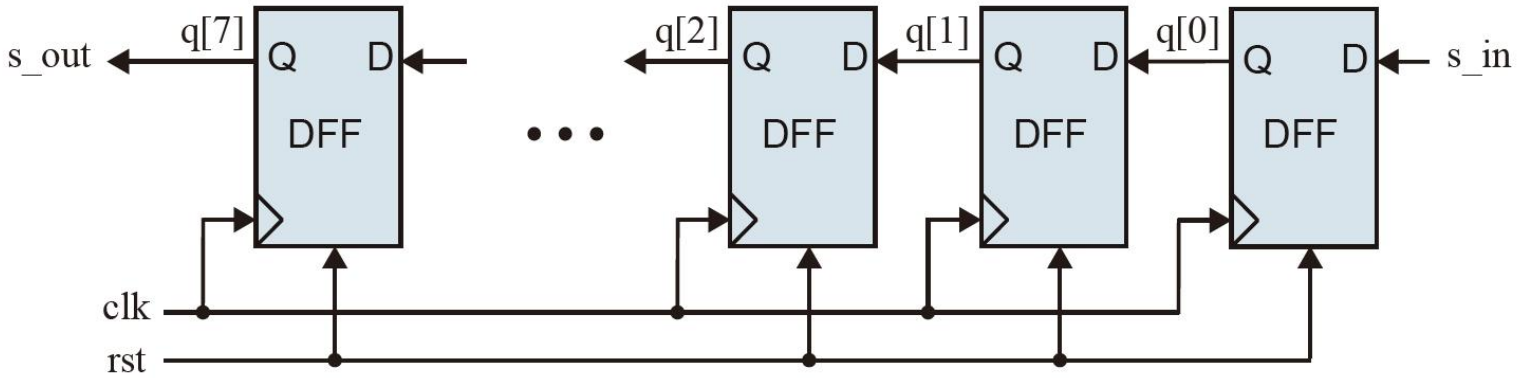
- Shift Register
  - 클록신호가 인가될 때마다 저장된 플립플롭에 저장된 데이터가 왼쪽 또는 오른쪽으로 이동되는 회로
  - 여러 개의 플립플롭이 직렬로 연결된 구조
  - 형태
    - 직렬입력 - 직렬출력 (Serial-In, Serial-Out)
    - 직렬입력 - 병렬출력 (Serial-In, Parallel-Out)
    - 병렬입력 - 직렬출력 (Parallel-In, Serial-Out)
    - 병렬입력 - 병렬출력 (Parallel-In, Parallel-Out)
    - 왼쪽 시프트, 오른쪽 시프트, 양방향 시프트
  - nonblocking 할당문, 시프트 연산자, 결합 연산자, 반복문 등으로 모델링

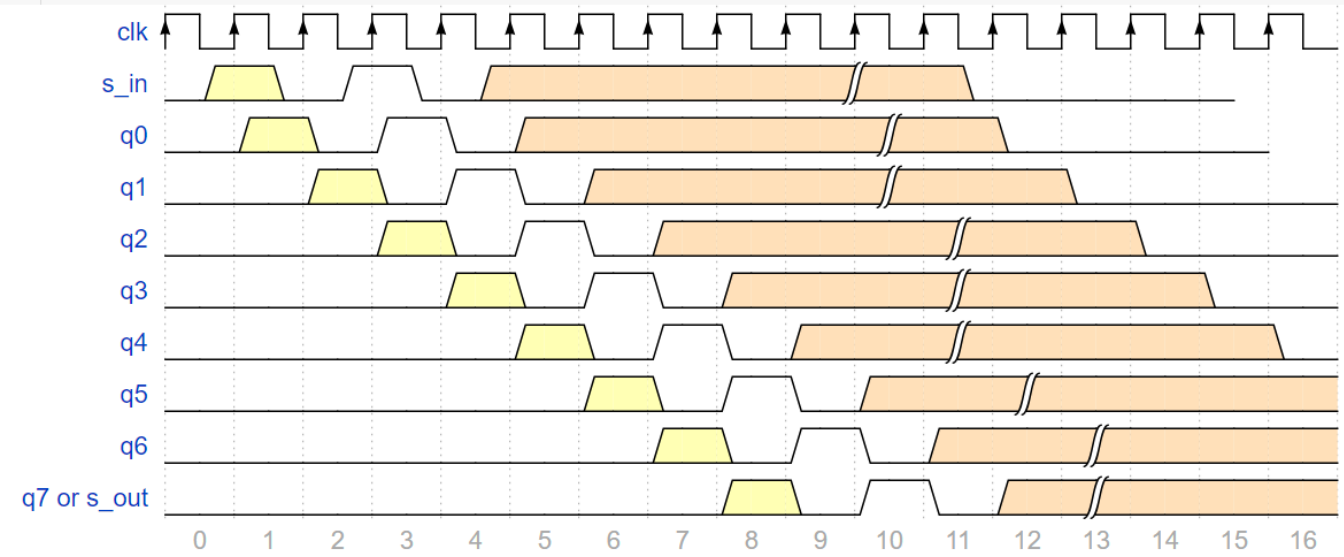
SoC 기본 순차논리 - 시프트 레지스터

기능	설명	활용 예시
SIPO	외부에서 직렬로 들어오는 데이터를 병렬로 변환하여 GPIO 출력으로 제공	SPI/I²C 등 직렬 인터페이스를 통해 데이터를 받아 GPIO 포트 확장
PISO	내부의 병렬 데이터를 직렬 형태로 변환하여 외부에 전송	여러 개의 스위치나 센서 등으로부터 읽은 GPIO 상태를 직렬 인터페이스를 통해 프로세서나 다른 장치로 전송

LAB : SIPO(Serial In Parallel Out) Shift Register

- 8비트 직렬입력-직렬/병렬 출력 시프트 레지스터

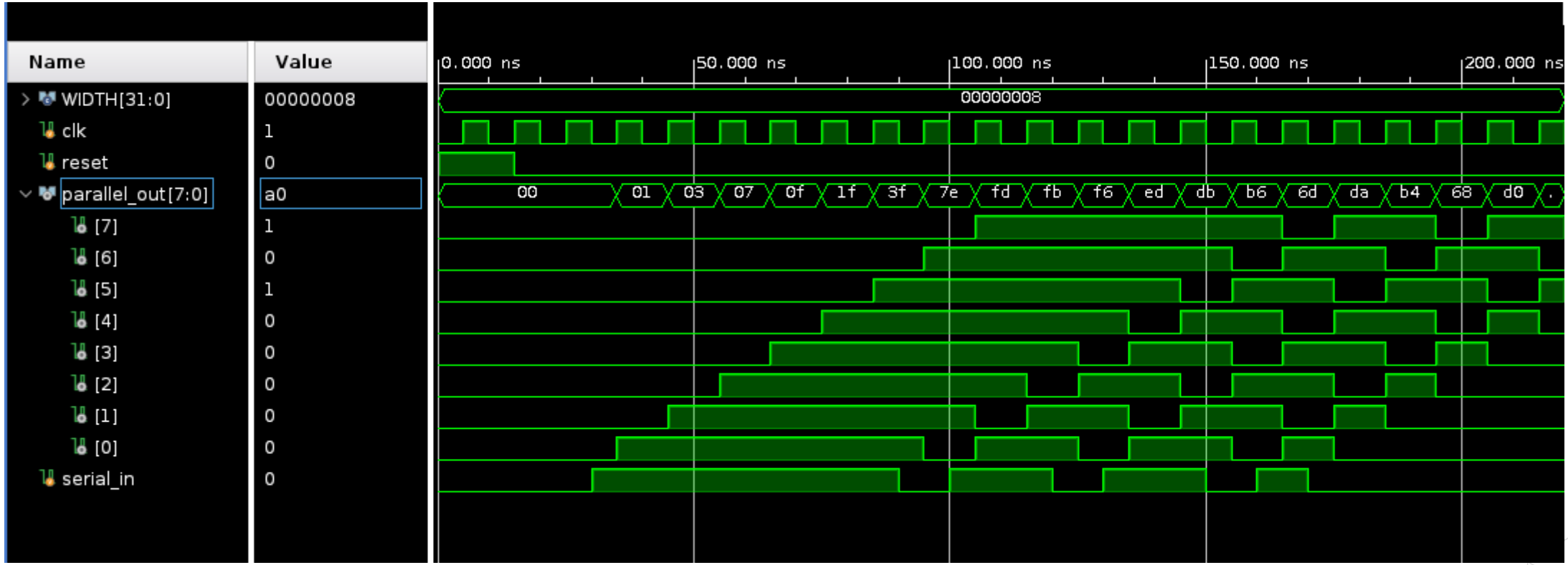




LAB : DUT

```
`timescale 1ns / 1ps

module shift_register_sipo #(
    parameter WIDTH = 8
)(
    input clk,
    input reset,
    input serial_in,
    output reg [7:0] parallel_out
);

always @(posedge clk or posedge reset) begin
    if (reset) begin
        parallel_out <= 8'b0;
    end else begin
        parallel_out <= {parallel_out[6:0], serial_in}; // 왼쪽으로 한 비트씩 시프트하고 가장 오른쪽에 입력 삽입
    end
end

endmodule
```

LAB : Testbench

```
`timescale 1ns / 1ps

module tb_shift_register_sipo;
localparam WIDTH = 8;

reg clk;
reg reset;
reg serial_in;
wire [WIDTH -1 :0] parallel_out;
// 모듈 인스턴스화
shift_register_sipo uut (
    .clk(clk),
    .reset(reset),
    .serial_in(serial_in),
    .parallel_out(parallel_out)
);

// 클럭 생성 (100MHz)
initial begin
    clk = 0;
    forever #5 clk = ~clk; // 주기 10ns (100MHz)
end

// 입력 데이터 및 리셋 신호 설정
initial begin
    reset = 1'b1; serial_in = 0;
    #15 reset = 1'b0;
    // random vector 생성 test
    repeat(WIDTH*2) begin
        @(negedge clk);
        serial_in = $random % 2; // 랜덤 비트 입력 (SoC 환경에서의 데이터 입력 상황 모사)
    end
    // 추가 클럭 사이클 진행 후 종료
    #50 $finish;
end

// 결과 모니터링
initial begin
    $monitor("time=%0t | reset=%b | serial_in=%b | parallel_out=%b", $time, reset, serial_in, parallel_out);
end

endmodule
```

LAB : Waveform